

REAL-TIME MONOCULAR DEPTH ESTIMATION ON EMBEDDED SYSTEMS

Cheng Feng¹, Congxuan Zhang^{2*}, Zhen Chen^{1,2*}, Weiming Hu³, and Liyue Ge¹

¹School of Instrumentation and Optoelectronic Engineering, Beihang University, China

²School of Measuring and Optical Engineering, Nanchang Hangkong University, China

³NLPR, Institute of Automation, Chinese Academy of Sciences, China

ABSTRACT

Depth sensing is of paramount importance for unmanned aerial and autonomous vehicles. Nonetheless, contemporary monocular depth estimation methods employing complex deep neural networks within Convolutional Neural Networks are inadequately expedient for real-time inference on embedded platforms. This paper endeavors to surmount this challenge by proposing two efficient and lightweight architectures, RT-MonoDepth and RT-MonoDepth-S, thereby mitigating computational complexity and latency. Our methodologies not only attain accuracy comparable to prior depth estimation methods but also yield faster inference speeds. Specifically, RT-MonoDepth and RT-MonoDepth-S achieve frame rates of 18.4&30.5 FPS on NVIDIA Jetson Nano and 253.0&364.1 FPS on Jetson AGX Orin, utilizing a single RGB image of resolution 640×192 . The experimental results underscore the superior accuracy and faster inference speed of our methods in comparison to existing fast monocular depth estimation methodologies on the KITTI dataset.

Index Terms— Deep Learning, Convolutional Neural Networks, Embedded Systems, Real-time, Depth Estimation

1. INTRODUCTION

Depth estimation is a crucial perception component for autonomous systems and can also support high-level vision tasks. For example, in the case of robot navigation, obstacle detection, or visual odometer often rely on depth information. While sensors such as LiDAR, stereo cameras, and millimeter-wave radar can provide accurate but sparse depth information, they can be expensive and bulky. In contrast, monocular depth estimation is a low-cost and easy-to-deploy approach to autonomous and intelligent systems that has recently advanced due to the progress in deep learning and reinforcement learning.

Considerable monocular depth estimation methods are proposed based on convolutional neural networks and different types of supervision. However, current state-of-the-art CNN-based monocular depth estimation methods [3, 4] have

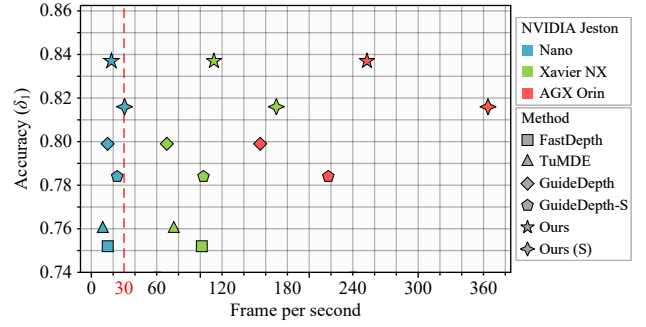


Fig. 1. Accuracy (in δ_1) vs. runtime (in FPS) on NVIDIA Jetson Nano, Xavier NX, and AGX Orin for various depth estimation algorithms on KITTI [1] dataset using the Eigen split [2]. The top right represents the desired characteristics of a depth estimation network design: high inference speed and high accuracy. The data comes from published papers or is measured using official implementation (online test, $batchsize = 1$).

primarily focused on achieving high accuracy, resulting in computationally intensive methods. Although these methods have significantly improved accuracy, achieving it comes at the cost of increased computational complexity, making them impractical for resource-constrained embedded devices. Since most embedded systems have limited computing resources and run multiple concurrent subtasks, achieving real-time performance with a limited computing budget while balancing latency and accuracy is a key challenge for practical deployment of monocular depth estimation in autonomous embedded systems.

Most recent works on efficient depth estimation approaches are devoted to improving the real-time performance of existing monocular depth estimation methods, focusing on the hardware-specific compilation, quantization, and model compression. In addition, some works [5, 6] use the existing lightweight backbone design to achieve faster execution, while others [7] design an efficient decoding structure to reduce latency.

This paper introduces a monocular depth estimation method suitable for embedded platforms that aims to balance

*Corresponding authors

depth estimation accuracy and latency for autonomous systems. Two efficient encoder-decoder network architectures, RT-MonoDepth and RT-MonoDepth-S, are proposed, which focus on high-accuracy with medium-latency design and medium-accuracy with low-latency design, respectively. The middle and low latency network designs, RT-MonoDepth and RT-MonoDepth-S, can perform real-time depth estimation on NVIDIA Jetson Nano¹, Xavier NX², and AGX Orin³ without post-processing. Fig. 1 illustrates that RT-MonoDepth can achieve reasonable accuracy and operate at over 253.0 frames per second (FPS) on AGX Orin and around 18.4 FPS on Nano. Moreover, RT-MonoDepth-S delivers solid performance at a high frame rate, operating at over 364.1 FPS on AGX Orin and around 30.5 FPS on Nano.

In summary, we propose two lightweight convolutional neural networks for monocular depth estimation. To the best of the authors' knowledge, our approach outperforms the related methods that target fast monocular depth estimation in terms of accuracy for the KITTI [1] benchmark while achieving the fastest inference speed.

2. RELATED WORK

Depth sensing from a monocular camera has been an active topic in the research of computer vision and robotics communities for over a decade. Eigen et al. [2] introduced the first CNN-based monocular depth estimation network, which utilizes a coarse-to-fine network with two stacked sub-networks. One sub-network predicts the global coarse depth map, and the other refines local details. In contrast, Laina et al. [8] suggested an end-to-end encoder-decoder architecture for monocular depth estimation, using a pre-trained ResNet [9] backbone as a feature extractor, and got comparable results to the depth sensor.

Godard et al. [10] proposed the first unsupervised training strategy, which utilizes the left-right consistency of stereo pairs to address the problem of CNN model training requiring a large amount of labeled data. Meanwhile, Zhou et al. [11] introduced an unsupervised framework that solves the depth estimation problem as a view synthesis problem by combining the depth model with a pose model. Furthermore, Godard et al. [12] suggested a self-supervised strategy by applying auto-masking to moving objects and using novel minimum reprojection loss to improve accuracy further, making Monodepth2 a significant baseline model.

Most recently, some studies aim to find a trade-off between accuracy and real-time performance. Initially, Wofk et al. [5] proposed FastDepth, an efficient method composed of a pre-trained MobileNetV2 [13] backbone with a lightweight decoder. To increase inference speed,

Wofk et al. [5] and Tu et al. [6] compiled their models with TVM [14], and employed refine network or reinforcement learning algorithm to reduce model size, respectively. In the latest research, Rudolph et al. [7] propose an efficient network, GuideDepth, composed of a DDRNet-23-slim [15] backbone and a lightweight decoder with a guided upsampling block. GuideDepth is compared with Wofk et al. [5] and Tu et al. [6] in the condition without post-processing and achieves the best result in both accuracy and inference speed on embedded platforms.

3. METHODOLOGY

Our proposed RT-MonoDepth framework is shown in Fig. 2. It consists of two parts: a pyramid encoder and a depth decoder. The pyramid encoder extracts high-level low-resolution features from the input image. These feature maps are then transmitted into the depth decoder, where they are gradually upsampled, fused, and used to predict the depth map at each scale. We pursue low latency and high-precision designs for both the pyramid encoder and the depth decoder in order to design a monocular depth estimation network that can run in real-time on embedded systems.

3.1. Lightweight Pyramid Encoder

The pyramid encoder in past monocular depth estimation methods usually employed a designed backbone network for image classification. ResNet [9] and MobileNet [13] were commonly used due to their powerful feature extraction ability and high accuracy. However, these structures are subject to limitations of complexity and latency, making them difficult to deploy to resource-constrained embedded devices. Moreover, it is hard to design a flexible decoder for low latency due to the fixed shape of the backbone network.

ResNet and MobileNet insert the batch normalization layer to accelerate convergence due to their complex topologies and deep layers. However, a small batch size brings instability to the model using batch normalization [16], and the capacity of GPU memory limits the dense regression task. Since we aim to design a simple, lightweight encoder, removing the normalization layer has no negative impact and can reduce computation slightly.

MobileNet reduces the computational complexity in theory by using depth-wise convolution instead of standard convolution. However, current implementations of deep learning frameworks cannot fully utilize the GPU without hardware support, making depth-wise convolution slow during both training and inference [17].

Taking the above two points into consideration, we construct a 4-layer pyramid encoder from scratch that targets low latency. Each layer features a *ConvBlock* which downsamples inputs and produces higher-level feature maps. The *ConvBlock* consists of three 3×3 standard convolution layers

¹<https://developer.nvidia.com/embedded/jetson-nano>

²<https://developer.nvidia.com/embedded/jetson-xavier-nx>

³<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>

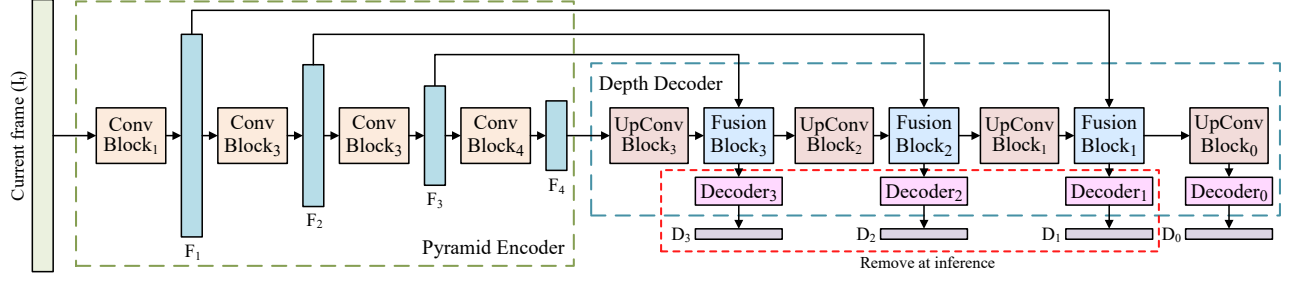


Fig. 2. Proposed RT-MonoDepth framework. The shape of input image is $H \times W \times 3$, symbol 'F' denotes feature maps in the pyramid encoder, symbol 'D' denotes predicted depth maps in each scale, the subscript number indicates their shape (e.g., $F_n: \frac{H}{2^n} \times \frac{W}{2^n} \times C_n$, $D_n: \frac{H}{2^n} \times \frac{W}{2^n} \times 1$). The Decoder₃, Decoder₂ and Decoder₁ can be removed when inferring.

without skip-connections or batch normalization layers, in order to minimize latency.

3.2. Simplified Depth Decoder

The function of the decoder is to upsample and fuse the output of the encoder to form a dense depth prediction. Our main objective is to determine how to design the three most critical parts of the depth decoder - upsampling layers, fusion layers, and prediction layers - in order to improve throughput and decrease the latency.

Upsampling: Upsampling operation is a fundamental design element of the low-latency depth decoder. Over the past few decades, various upsampling methods have been proposed, such as unpooling, transpose convolution, interpolation, and interpolation combined with convolution. FastDepth [5] explored the impact of different upsampling methods on depth performance. They compared different combinations of upsampling methods and ultimately proposed an upsampling method called NNConv5, which consists of 5×5 depth-wise separable convolution followed by nearest-neighbor interpolation with a scale factor of 2. To achieve an even faster inference speed, we have substituted NNConv5 with NNConv3. Each NNConv3 *UpConvBlock* performs 3×3 standard convolution and reduces the number of output channels by $\frac{1}{2}$ relative to the number of input channels, and followed a nearest-neighbor interpolation that doubles the resolution of intermediate feature maps.

Fusion: Fusing features of different scales is an essential means to improve performance in many works [18]. Low-level features with high resolution contain more spatial information but less semantic information and more noise. On the other hand, high-level features with low resolution contain more semantic information but less spatial information. Two standard fusion methods in deep learning practice are element-wise addition and concatenation. Monodepth2 [12] gradually concatenates upper-level feature maps to lower levels, while FastDepth [5] gradually adds upper-level feature maps to the next level. In contrast, GuideDepth [7] introduces the guided upsampling block, which uses the input im-

age as guidance to upsample feature maps without previous level feature maps. To balance latency and accuracy, we perform element-wise addition at the first and second layers to accelerate feature fusion, and using concatenate operation at the third layer of the depth decoder for better final accuracy.

Prediction: Our model sets one prediction decoder at each scale. Each decoder consists of two 3×3 convolution layers, followed by a leakyReLU and a sigmoid function as activating functions, respectively. The decoders receive the fused feature maps and yield depth predictions at each scale for self-supervision. To minimize latency, we do not pass the intermediate depth prediction to the next level. This allows us to disable the first three decoders to speed up inference.

4. EXPERIMENTS

In this section, we present experimental results to demonstrate the effectiveness of our approach. We first introduce the details of our implementation and then evaluate our method against existing high-precision or low-latency methods on the KITTI [1] benchmark for monocular depth estimation. We also provide ablation studies of our framework based on accuracy and latency metrics. Finally, we discuss the limitations of our approach and suggest future directions for improvement.

4.1. Implementation details

Hardware Platforms: We train our networks on a single NVIDIA RTX 3090 GPU with 9 GB of GPU memory, and evaluate the real-time performance on three edge computing embedded systems: the NVIDIA Jetson Nano, Xavier NX, and AGX Orin. The Jetson Nano and Xavier NX are low-power embedded devices, while the AGX Orin is the most powerful platform deployed in autonomous systems. We report results for Jetson Nano in 10W power mode and Xavier NX in 15W power mode, using the same configurations as described in [7]. Additionally, we present detailed results for the AGX Orin to highlight our advantage compared to state-of-the-art fast depth estimation methods.

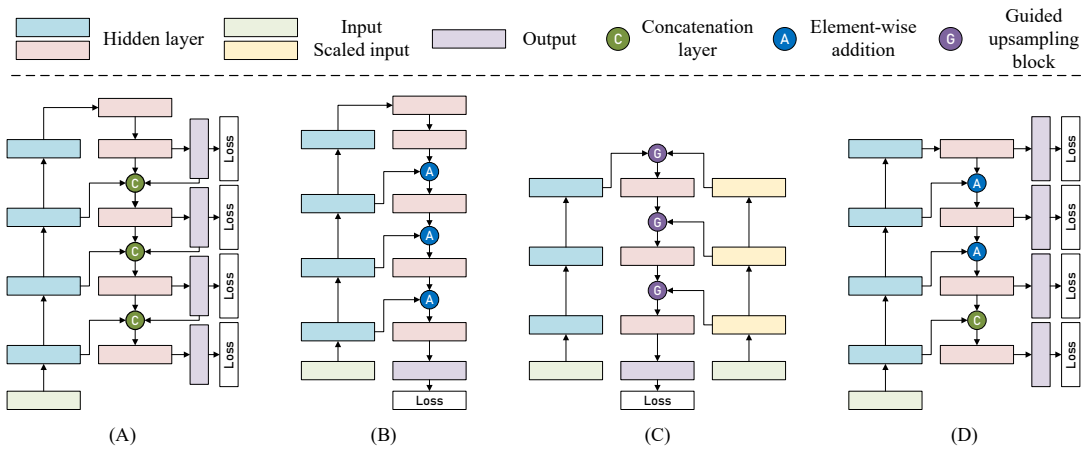


Fig. 3. Feature fusion methods of different approaches. (A) Monodepth2 [12]; (B) FastDepth [5]; (C) GuideDepth [7]; (D) Our model.

Dataset: We train and evaluate our networks on the KITTI [1] dataset, following the train/test data split described in [2]. We remove static frames [11] during training, and clip the depth to 80m per standard practice [10] during evaluating.

Training Details: Our method is implemented in PyTorch using 32-bit floating-point precision. We use the AdamW optimizer with a learning rate of 1×10^{-4} and a batch size of 8. We train for 20 epochs, reducing the learning rate by 10 after 15 epochs. Data augmentation and self-supervised strategy during training are the same as in Monodepth2 [12].

Testing Details: There are two standard testing methods in deep learning: offline and online. Offline testing is used when real-time requirements are not necessary, allowing multiple data ($batchsize = N$) to be processed in parallel. In contrast, online testing only allows one data ($batchsize = 1$) to be processed at a time to test real-time performance. For a fair comparison, all inference speeds in our experiments are reported on models converted to 16-bit floating-point precision TensorRT⁴ with online testing. Unlike [7], we warm up each method by running it 1000 times and then measure the time consumption with an average test time of 5000 iterations (10 warm-up and 200 testing in [7]).

Evaluation Metrics: We use standard six metrics used in used in prior work [12] to compare various methods. Let $\hat{d}_i$ be the pixel of prediction depth map $\hat{d}$, and d_i be the pixel in the ground-truth d , and N be the total number of pixels. Then we use the following evaluation metrics: absolute relative error (AbsRel): $\frac{1}{N} \sum \frac{|d_i - \hat{d}_i|}{d_i}$; square relative error (SqRel): $\frac{1}{N} \sum \frac{|d_i - \hat{d}_i|^2}{d_i}$; root mean square error (RMSE): $\sqrt{\frac{1}{N} \sum |d_i - \hat{d}_i|^2}$; root mean square logarithmic error (RMSE_{log}): $\sqrt{\frac{1}{N} \sum |\log_{10} d_i - \log_{10} \hat{d}_i|^2}$; threshold accuracy (δ_j): % of d_i s.t. $\max(\frac{d_i}{\hat{d}_i}, \frac{\hat{d}_i}{d_i}) = \delta < thr$ for

$$thr = 1.25, 1.25^2, 1.25^3.$$

4.2. Final Results and Comparison With Prior Work

Table 1 summarizes the results of our methodology compared to existing high-precision or low-latency methods. The 'D' symbol in the Data column indicates methods that use depth supervision during training, 'M' denotes self-supervised training on monocular video, and 'MS' represents models trained with stereo video. We also include results for Monodepth2 with ImageNet pretraining, indicated by a checkmark in the pre-train column.

We compare our model to low-latency methods such as FastDepth [5], TuMDE [6], and GuideDepth [7], as well as the classical method Monodepth2 [12]. We also evaluate a smaller model called RT-MonoDepth-S, which drops the *FusionBlock* and reduces the layers of *ConvBlock* in the original model, resulting in about 6% less accuracy but 47% faster inference time than RT-MonoDepth.

As shown in Table 1, RT-MonoDepth achieves an average of 5% better performance with 53% fewer parameters and an average of 60% higher FPS. The RT-MonoDepth-S lite version performs an average of 3% better than GuideDepth with 75% fewer parameters and an average of 63% higher FPS. Compared to the classical method Monodepth2 without pre-training, RT-MonoDepth slightly outperforms Monodepth2 in all metrics. However, there is still a significant performance gap between our method and pre-trained Monodepth2.

As a result of our efficient and lightweight architecture design, RT-MonoDepth can achieve up to 18.4 FPS on the Jetson Nano and 253.0 FPS on the AGX Orin. RT-MonoDepth-S delivers up to 30.5 FPS on the NVIDIA Jetson Nano and up to 364.1 FPS on the AGX Orin. The low latency makes our methods suitable for deployment on autonomous robot platforms with high real-time performance requirements. As far as we know, our method is the fastest non-pruned monocular

⁴<https://developer.nvidia.com/tensorrt>

Table 1. Quantitative results of depth estimation on the KITTI dataset (Eigen split, 640×192 , 0-80m).

methods	Data	pre-train	#para (M)	AbsRel ↓	SqRel ↓	RMSE ↓	RMSE _{log} ↓	$\delta < 1.25 \uparrow$	$\delta < 1.25^2 \uparrow$	$\delta < 1.25^3 \uparrow$	FPS↑ (Nano)	FPS↑ (Xavier)	FPS↑ (Orin)
FastDepth [5]	D	•	4.0	0.168	-	5.839	-	0.752	0.927	0.977	15.0	101.4	-
TuMDE [6]	D	•	5.7	<u>0.150</u>	-	5.801	-	0.760	0.930	0.980	10.5	75.7	-
GuideDepth [7]	D	•	5.8	0.142	-	5.194	-	0.799	0.941	0.982	15.2	69.3	155.0
GuideDepth-S [7]	D	•	5.7	0.142	-	<u>5.480</u>	-	<u>0.784</u>	<u>0.936</u>	<u>0.981</u>	25.2	102.9	217.5
Monodepth2 [12]	M		14.3	0.132	1.044	5.142	0.210	0.845	0.948	0.977	-	-	142.3
Monodepth2 [12]	M	•	14.3	0.115	0.903	4.863	0.193	0.877	0.959	0.981	-	-	142.3
RT-MonoDepth	M		2.8	<u>0.125</u>	<u>0.959</u>	<u>4.985</u>	<u>0.202</u>	<u>0.857</u>	<u>0.952</u>	<u>0.979</u>	<u>18.4</u>	<u>112.5</u>	<u>253.0</u>
RT-MonoDepth-S	M		1.2	0.132	0.997	5.262	0.214	0.840	0.946	0.977	30.5	169.8	364.1
Monodepth2 [12]	MS		14.3	<u>0.127</u>	1.031	5.266	0.221	0.836	0.943	0.974	-	-	142.3
Monodepth2 [12]	MS	•	14.3	0.106	0.818	4.750	0.196	0.874	0.957	0.979	-	-	142.3
RT-MonoDepth	MS		2.8	<u>0.127</u>	<u>0.995</u>	<u>5.153</u>	<u>0.217</u>	<u>0.837</u>	<u>0.944</u>	<u>0.975</u>	<u>18.4</u>	<u>112.5</u>	<u>253.0</u>
RT-MonoDepth-S	MS		1.2	0.135	1.040	5.413	0.229	0.816	0.936	0.973	30.5	169.8	364.1

depth estimation method on embedded systems.

Fig. 4 presents the qualitative results of our RT-MonoDepth method, the state-of-the-art fast monocular depth estimation method GuideDepth, and the classical method Monodepth2 (without pre-training). As shown in Fig. 4, our method outperforms GuideDepth significantly and can provide more effective object reconstruction than Monodepth2, especially in the edge regions.

4.3. Ablation Studies

We investigate the design space of the RT-MonoDepth framework and perform ablation studies on supervision scales, pyramid encoder levels and feature fusions. All ablation studies are tested on Jetson Nano at a resolution of 640×192 .

Supervision scales: Monodepth2 [12], FastDepth [5], and GuideDepth [7] gradually merge feature maps in the decoder to output a full-resolution depth map at the last level. Monodepth2 [12] predicts depth maps and applies supervision signals at different scales, and these intermediate depth maps are merged upward and cannot be ignored during inference. FastDepth [5] and GuideDepth [7] only yield depth maps at the last level and do not apply additional intermediate supervision signals. Table 2 summarizes the results of four variants that use 1, 2, 3, and 4 layers of supervision. The results indicate that deeper depth supervision leads to consistently better performance. Unlike Monodepth2, the intermediate predictions of RT-MonoDepth will not affect the final real-time performance as they can be ignored during inference.

Feature Pyramid Levels: The feature pyramid levels determine the minimum resolution at which the decoder starts, which is a critical parameter of coarse-to-fine architectures. To fully utilize the employed backbone network, Monodepth2 [12] and FastDepth [5] extract the feature map with a minimum resolution of $\frac{1}{32}$. On the other hand, GuideDepth [7] only extracts the feature map with a minimum resolution of $\frac{1}{8}$ to achieve faster inference speed. Table 2 summarizes the results of four variants that use 2, 3, 4, and 5 levels of the feature pyramid, which extract the feature map with a minimum resolution of $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$, and $\frac{1}{32}$, respectively.

Table 2. Ablation experiments. **Bold:** Final settings.

Ablation	Parameter	AbsRel ↓	RMSE ↓	$\delta < 1.25 \uparrow$	FPS↑
Supervision Scales	1	0.128	5.227	0.834	18.4
	2	0.128	5.157	0.834	18.4
	3	0.128	5.158	0.836	18.4
	4	0.127	5.153	0.837	18.4
Encoder Levels	2	0.146	5.357	0.814	24.3
	3	0.133	5.207	0.828	20.9
	4	0.127	5.153	0.837	18.4
	5	0.123	5.080	0.837	16.2
Fusion Methods	+++	0.132	5.257	0.834	19.3
	++c	0.127	5.153	0.837	18.4
	+cc	0.126	5.139	0.837	17.6
	ccc	0.126	5.086	0.838	15.8

The results indicate that more levels of the feature pyramid provide higher precision but worse real-time performance. A pyramid level of 4 strikes a balance between accuracy and latency.

Feature Fusions: We evaluate the accuracy and latency of the model under different feature fusion methods. Table 2 summarizes the results of four methods that use different feature fusion strategies. The symbol '+' represents element-wise addition, and 'c' stands for concatenation. The results demonstrate that more concatenation operations provide higher precision but worse real-time performance. The '++c' method strikes a balance between accuracy and latency.

4.4. Comparison of inferring speed

We conduct various speed tests of RT-MonoDepth and the state-of-the-art fast monocular depth estimation method GuideDepth on the AGX Orin and Nano using an image resolution of 640×192 . The results are presented in Fig. 5. In the full power mode (Orin Max and Nano Max), our method outperforms GuideDepth significantly. With GPU optimization, the advantages of our method become more prominent. Furthermore, we set AGX Orin to work in 15W mode to simulate resource-constrained scenarios where a single device runs multiple tasks in parallel. The results demonstrate that

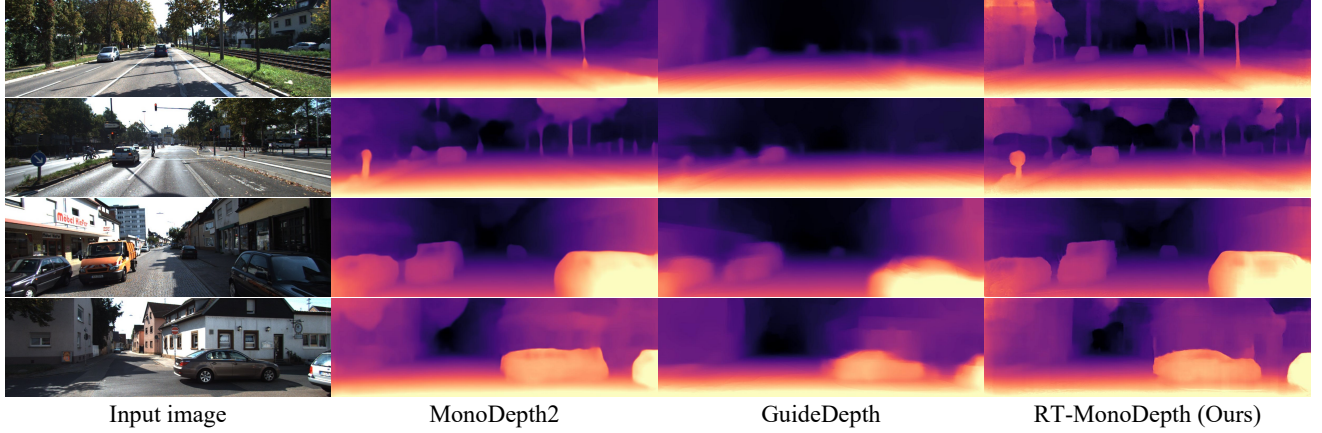


Fig. 4. Qualitative results of monocular depth estimation comparing RT-MonoDepth with Monodepth2 and GuideDepth on the KITTI dataset. Our method provides cleaner boundaries and more effective object reconstruction than the other methods.

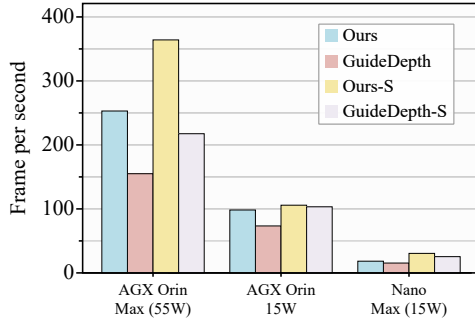


Fig. 5. Comparison results of inferencing speed comparing RT-MonoDepth with GuideDepth. Our method provides faster inferencing speed than the GuideDepth.

our method is faster than GuideDepth, even under conditions of limited computing resources.

5. LIMITATION

Table 3 illustrates the performance of RT-MonoDepth across varying input resolutions. While our method demonstrates promising outcomes at lower resolutions, its efficacy diminishes with high-resolution inputs. Specifically, our model, when trained with high-resolution images, exhibits an approximate 4% reduction in average performance compared to the low-resolution counterpart. This performance decline aligns with findings in existing research [19, 4], indicating that monocular depth estimation models heavily rely on learned image edge features. Higher-resolution images necessitate a model with an encoder boasting an expanded receptive field and complexity for optimal learning. Given that our model employs a lightweight, simplistic encoder, accurate learning becomes challenging with high-resolution inputs, contributing to the observed performance degradation.

Table 3. Quantitative results of RT-MonoDepth with different resolution images on the KITTI dataset (Eigen split, 0-80m).

Resolution	Data	AbsRel ↓	RMSE ↓	$\delta < 1.25 \uparrow$
640×192	M	0.125	4.985	0.857
	MS	0.126	5.157	0.836
1024×320	M	0.139	5.108	0.837
	MS	0.136	5.182	0.825

6. CONCLUSION

In this work, we focus on designing low-latency monocular depth estimation networks for resource-constrained robot systems on embedded platforms. We propose efficient network architectures, RT-MonoDepth and RT-MonoDepth-S, with low complexity and low-latency encoder-decoder design, achieving high frame rates on Jetson Nano, Xavier NX, and AGX Orin while maintaining comparable accuracy with related architectures on the KITTI dataset. Our method is the fastest no-pruning monocular depth estimation method to date. We hope our methods can be practically applied to real-world autonomous driving and robotics. For future work, we aim to improve the accuracy of high-resolution input and further reduce runtime.

7. ACKNOWLEDGE

This work was supported in part by the National Natural Science Foundation of China under Grant 62222206 and Grant 62272209; in part by the Technology Innovation Guidance Program of Jiangxi Province under Grant 20212AEI91005; in part by the Key Research and Development Program of Jiangxi Province under Grant 20232BBE50006; in part by the Major Research and Development Project of Jiangxi Province 20232ACC01007; and in part by the Open Fund for Key Laboratory of Image Processing and Pattern Recognition of Jiangxi Province under Grant ET202208305.

8. REFERENCES

- [1] Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun, "Vision meets robotics: The KITTI dataset," *Int. J. Robotics Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [2] David Eigen, Christian Puhrsch, and Rob Fergus, "Depth map prediction from a single image using a multi-scale deep network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2366–2374.
- [3] Guanglei Yang, Hao Tang, Mingli Ding, Nicu Sebe, and Elisa Ricci, "Transformer-based attention networks for continuous pixel-wise prediction," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 16249–16259.
- [4] S. Mahdi H. Miangoleh, Sebastian Dille, Long Mai, Sylvain Paris, and Yagiz Aksoy, "Boosting monocular depth estimation models to high-resolution via content-adaptive multi-resolution merging," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9685–9694.
- [5] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6101–6108.
- [6] Xiaohan Tu, Cheng Xu, Siping Liu, Renfa Li, Guoqi Xie, Jing Huang, and Laurence Tianruo Yang, "Efficient monocular depth estimation for edge devices in internet of things," *IEEE Trans. Ind. Informatics*, vol. 17, no. 4, pp. 2821–2832, 2021.
- [7] Michael Rudolph, Youssef Dawoud, Ronja G ldenring, Lazaros Nalpantidis, and Vasileios Belagiannis, "Lightweight monocular depth estimation through guided decoding," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2344–2350.
- [8] Iro Laina, Christian Rupprecht, Vasileios Belagiannis, Federico Tombari, and Nassir Navab, "Deeper depth prediction with fully convolutional residual networks," in *International Conference on 3D Vision (3DV)*, 2016, pp. 239–248.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [10] Cl ment Godard, Oisin Mac Aodha, and Gabriel J. Brostow, "Unsupervised monocular depth estimation with left-right consistency," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6602–6611.
- [11] Tinghui Zhou, Matthew Brown, Noah Snavely, and David G. Lowe, "Unsupervised learning of depth and ego-motion from video," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6612–6619.
- [12] Cl ment Godard, Oisin Mac Aodha, Michael Firman, and Gabriel J. Brostow, "Digging into self-supervised monocular depth estimation," in *IEEE/CVF International Conference on Computer Vision, ICCV*, 2019, pp. 3827–3837.
- [13] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [14] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy, "TVM: an automated end-to-end optimizing compiler for deep learning," in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018, pp. 578–594.
- [15] Yuanduo Hong, Huihui Pan, Weichao Sun, and Yisong Jia, "Deep dual-resolution networks for real-time and accurate semantic segmentation of road scenes," *arXiv preprint*, arXiv:2101.06085, 2021.
- [16] Yuxin Wu and Kaiming He, "Group normalization," in *European Conference on Computer Vision (ECCV)*, 2018, vol. 11217, pp. 3–19.
- [17] Zheng Qin, Zhaoning Zhang, Dongsheng Li, Yiming Zhang, and Yuxing Peng, "Diagonalwise refactorization: An efficient training method for depthwise convolutions," in *International Joint Conference on Neural Networks (IJCNN)*, 2018, pp. 1–8.
- [18] Zhenli Zhang, Xiangyu Zhang, Chao Peng, Xiangyang Xue, and Jian Sun, "Exfuse: Enhancing feature fusion for semantic segmentation," in *European Conference on Computer Vision (ECCV)*, 2018, vol. 11214, pp. 273–288.
- [19] Junjie Hu, Yan Zhang, and Takayuki Okatani, "Visualization of convolutional neural networks for monocular depth estimation," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 3868–3877.